Assignment #2

Analyze and compare the performance of two all-pairs shortest path algorithms: Johnson's Algorithm and Floyd–Warshall Algorithm.Evaluate them on the following
graphs:
i) Sparse Graph: 45 vertices, 150–200 randomly generated edges.
ii) Dense Graph: 45 vertices, complete graph (990 edges).

Date of Performance: 17/8/26

Date of Submission: 17/8/26

Student ID: 00724105101024

Name: Sakif Ahmed Rafi

Group: A1

Objective :

The task is to generate both sparse and dense graphs, run Floyd–Warshall and Johnson's algorithms on each graph, and then analyze and compare their performance and efficiency.

Tasks:

1. Generate Sparse graph of 45 vertices and 200 edges
   Generate Dense graph of 45 edges and 990 edges

2. Algorithm Implementation
   Implement Floyd Warshall and Johnson's algorithm

3. Performance Measurement:
   For each run measure
   time (s)
   Estimated energy consumption (J)
   Energy (J) = CPU's approximate average power x Execution Time (s)
   Peak memory usage (KB)
   Estimated CO_2 emissions (Bangladesh) Energy (kWh) = Energy (J) / 3,600,000 CO_2(kg) = Energy (kWh) x 0.62

Algorithms:

```
//00724105101024

#include <windows.h>
#include <psapi.h>
#include <stdio.h>

#pragma comment(lib, "Psapi.lib")

// Replace with your CPU's approximate average power (Watts)
#define CPU_POWER_WATTS 65.0
// Bangladesh grid emission factor (kg CO₂ / kWh)
#define BD_EMISSION_FACTOR 0.62

#include <bits/stdc++.h>
using namespace std;

const int N = 45;
const int INF = 1e9;
```

```
vector<pair<pair<int,int>,int>> edge_list;
vector<vector<pair<int,int>>> adj_list;
vector<int> dist;
vector<int> h;

void generate_sparse_graph(){

   edge_list.clear();

   int current,neighbour,weight;

   for(int i=0; i<200; i++){
      current = rand()%N;
      neighbour = rand()%N;
      weight = rand()%20+1;

      edge_list.push_back({{current,neighbour},weight});
   }
}

void generate_dense_graph(){

   edge_list.clear();

   int weight;

   for(int i=0; i<N; i++){
      for(int j=i+1; j<N; j++){

         weight = rand()%20+1;

         edge_list.push_back({{i,j},weight});
      }
   }
}

void floyd_warshall(int n){

   vector<int> vec(n,INF);
   vector<vector<int>> adj_matrix(n,vec);

   for(auto [x,y] : edge_list){
      adj_matrix[x.first][x.second] = y;
   }

   for(int i=0; i<n; i++){
      adj_matrix[i][i] = 0;
   }

   for(int i=0; i<n; i++){
      for(int j=0; j<n; j++){
```

```
            if(j==i) continue;

            for(int k=0; k<n; k++){

                if(k==j) continue;

                if(adj_matrix[j][i]==INF || adj_matrix[i][k]==INF)
                    continue;

                if(adj_matrix[j][i]+adj_matrix[i][k] < adj_matrix[j][k]){
                    adj_matrix[j][k] =
                        adj_matrix[j][i]+adj_matrix[i][k];
                }
            }
        }
    }
}

void bellman_ford(int n, int src){

    h.clear();
    h.resize(n,INF);

    h[src] = 0;

    for(int i=0; i<n-1; i++){

        for(auto [x,y] : edge_list){

            if(h[x.first]==INF) continue;

            if(h[x.first]+y < h[x.second])
                h[x.second] = h[x.first]+y;
        }
    }
}

void dijkstra(int src){

    dist.clear();
    dist.resize(N,INF);

    dist[src] = 0;

    priority_queue<
        pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>
    > pq;

    pq.push({dist[src],src});
```

```
    pair<int,int> current;

    while(!pq.empty()){

        current = pq.top();
        pq.pop();

        if(current.first>dist[current.second])
            continue;

        for(auto [neighbour,weight] : adj_list[current.second]){

            if(current.first+weight < dist[neighbour]){

                dist[neighbour] = current.first+weight;

                pq.push({dist[neighbour],neighbour});
            }
        }
    }
}

void johnson(int n){

    int new_src = n;

    for(int i=0; i<n; i++){
        edge_list.push_back({{new_src,i},0});
    }

    bellman_ford(n+1,new_src);

    adj_list.clear();
    adj_list.resize(n);

    for(auto [x,y] : edge_list){

        int current = x.first;
        int neighbour = x.second;
        int weight = y;

        if(current==new_src)
            continue;

        int new_weight = weight+h[current]-h[neighbour];

        adj_list[current].push_back({neighbour,new_weight});
    }

    for(int i=0; i<n; i++){
```

```
        dijkstra(i);

        for(int j=0; j<n; j++){

            if(dist[j]==INF)
                continue;

            dist[j] = dist[j]-h[i]+h[j];
        }
    }
}

int main() {

    LARGE_INTEGER freq, start, end;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&start);

    // -------- Your algorithm here --------

    generate_sparse_graph();
    floyd_warshall(N);

    edge_list.clear();

    generate_sparse_graph();
    johnson(N);

    edge_list.clear();

    generate_dense_graph();
    floyd_warshall(N);

    edge_list.clear();

    generate_dense_graph();
    johnson(N);

    // ------------------------------------

    QueryPerformanceCounter(&end);

    double elapsed = (double)(end.QuadPart - start.QuadPart) / freq.QuadPart;
    double energy = CPU_POWER_WATTS * elapsed;        // Joules
    double energy_kWh = energy / 3.6e6;               // kWh
    double co2 = energy_kWh * BD_EMISSION_FACTOR;     // kg CO₂

    // ----- Memory usage -----
    PROCESS_MEMORY_COUNTERS_EX pmc;
    if (GetProcessMemoryInfo(GetCurrentProcess(),
(PROCESS_MEMORY_COUNTERS*)&pmc, sizeof(pmc))) {
        SIZE_T peakMemUsed = pmc.PeakWorkingSetSize; // Peak RAM usage
```

```
    printf("Execution time: %.6f seconds\n", elapsed);
    printf("Estimated energy consumption: %.2f Joules\n", energy);
    printf("Peak Memory Usage: %zu KB\n", peakMemUsed / 1024);
    printf("Estimated CO2 emissions (Bangladesh): %.8f kg\n", co2);
  } else {
    printf("Failed to get memory info.\n");
  }

  return 0;
}
```

Algorithm	Graph Type	Nodes	Edges	Execution Time(s)	Energy (J)	Peak Memory (KB)	CO2 Emissions (kg, BD)
Floyd Warshall	Sparse	45	200	0.000214600	0.01394900	4548	0.000000002
Johnson	Sparse	45	200	0.001824300	0.11857950	4576	0.000000020
Floyd Warshall	Dense	45	990	0.000327800	0.02130700	4652	0.000000004
Johnson	Dense	45	990	0.002183700	0.14194050	4812	0.000000024

Discussion

- **Execution Speed:** From the results, Floyd–Warshall performs faster than Johnson's algorithm for both the sparse and dense graphs. This is interesting because Johnson's algorithm is theoretically expected to perform better on sparse graphs.

- **Theory vs. Practical Performance**: The theoretical advantage of Johnson's algorithm becomes more useful as the graph size increases. In our experiment, the graph has only 45 nodes, so Floyd–Warshall can complete its simple three nested loops very quickly. Johnson's algorithm, on the other hand, has additional overhead from Bellman–Ford, priority queues, adjacency lists, and running Dijkstra's algorithm from every node. For such a small graph, this extra work makes Johnson slower in practice.

- **Energy and Memory Usage**: Since Floyd–Warshall takes

less time to finish, it also uses less energy and produces slightly lower CO_2 emissions in our experiment. Its memory usage is also slightly lower than Johnson's algorithm for both graph types.

Conclusion

For the 45-node graphs used in this experiment, Floyd–Warshall performed better overall in terms of execution time, energy consumption, and simplicity. Although Johnson's algorithm has a theoretical advantage for large and sparse graphs, its additional processing overhead makes it less suitable for a small graph like this one. Therefore, for this experiment, Floyd–Warshall is the more practical choice, while Johnson's algorithm would become more attractive as the graph size grows, particularly when the graph is sparse.